

## Smart QR-Based Attendance Management System

### Complete Project Documentation v2.1 (FINAL - Production-Ready)

**Project Type:** B.Tech Full-Stack Development Project

**Academic Level:** 2nd Year, 2nd Semester

**Technology Stack:** Java (Spring Boot) + React.js (Web)

**Duration:** 8 Weeks

**Document Version:** 2.1 (FINAL)

**Last Updated:** January 10, 2026

**Review Status:** Professor Approved ☒

---

### PROFESSOR FEEDBACK ADDRESSED (v2.1)

Version 2.0 → 2.1 Changes:

#### ☒ Fixed HTTPS Geolocation Issue

- Added Ngrok setup instructions
- Self-signed SSL certificate guide
- Development vs production environment handling

#### ☒ Fixed Mobile Data IP Conflict

- UI warning: "Connect to College Wi-Fi, disable Mobile Data"
- Helpful error messages
- Pre-scan connectivity check

#### ☒ Fixed Silent Registration Device Limit

- Removed auto-registration
- Added explicit modal with slot counter
- Device management UI

#### ☒ Fixed Notification Spam

- Removed active alerts for low confidence
- Yellow highlighting in reports instead
- Passive teacher review system

---

### Table of Contents

1. [Executive Summary](#)
2. [Development Environment Setup](#)
3. [Core Features](#)

4. [Validation Pipeline](#)
  5. [Database Design](#)
  6. [Implementation Guide](#)
  7. [Testing Strategy](#)
  8. [Deployment](#)
  9. [Timeline](#)
  10. [Troubleshooting Guide](#)
- 

## 1. Executive Summary

A realistic, implementable QR-based attendance system with:

- Time-rotating QR codes (20s expiry)
- IP subnet validation (campus network check)
- GPS soft-fail (30m ideal, 100m max)
- Device registration (explicit user consent)
- **8-week realistic scope**

### Security Layers:

1. JWT Authentication
2. Device Token
3. QR Token Expiry
4. Enrollment Verification
5. IP Subnet Validation
6. GPS Validation (soft-fail)

**What We WILL Build:** Core attendance system

**What We WON'T Build:** ML/Analytics/Appeals (future scope)

---

## 2. Development Environment Setup

### 2.1 The HTTPS Geolocation Problem (CRITICAL)

**Issue:** navigator.geolocation only works on HTTPS or localhost.

**Scenario:**

✓ Works: http://localhost:3000 (development)

✗ Fails: http://192.168.1.50:3000 (phone testing)

✓ Works: <https://192.168.1.50:3000> (with SSL cert)

✓ Works: <https://xyz.ngrok.io> (tunneling)

## 2.2 Solution 1: Ngrok (RECOMMENDED for Development)

### Setup Steps:

# Install Ngrok

# Download from <https://ngrok.com/download>

# Or use npm

```
npm install -g ngrok
```

# Start React dev server

```
npm start # Runs on http://localhost:3000
```

# In another terminal, create HTTPS tunnel

```
ngrok http 3000
```

# Output:

# Forwarding <https://a1b2c3d4.ngrok.io> -> <http://localhost:3000>

**Students access:** <https://a1b2c3d4.ngrok.io> (HTTPS ✓, Geolocation works ✓)

### Pros:

- ✓ No certificate setup needed
- ✓ Works immediately
- ✓ Can share with team members anywhere

### Cons:

- ⚠ URL changes on each restart (free tier)
- ⚠ Requires internet connection

### package.json script:

```
{  
  "scripts": {  
    "start": "react-scripts start",  
    "dev:mobile": "ngrok http 3000"
```

```
}  
}
```

### 2.3 Solution 2: Self-Signed SSL Certificate (Local Network)

#### For Testing on Local Wi-Fi:

# Generate self-signed certificate

```
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -days 365 -nodes
```

# Update package.json

```
{  
  "scripts": {  
    "start:https": "HTTPS=true SSL_CERT_FILE=cert.pem SSL_KEY_FILE=key.pem react-scripts start"  
  }  
}
```

# Start with HTTPS

```
npm run start:https
```

# Access: <https://192.168.1.50:3000>

**Note:** Students will see "Not Secure" warning, but must click "Proceed anyway" for testing.

### 2.4 Production Deployment (College Server)

#### For Final Deployment:

- Use Let's Encrypt (free SSL certificate)
- Domain: attendance.yourcollege.edu
- HTTPS by default

#### Setup:

# Install Certbot

```
sudo apt install certbot
```

# Get free SSL certificate

```
sudo certbot --nginx -d attendance.yourcollege.edu
```

### 2.5 Development Workflow

#### Week 1-6 (Development):

Local: <http://localhost:3000>

Testing: <https://xyz.ngrok.io> (use Ngrok)

### **Week 7-8 (Final Testing):**

Deploy to college server with proper SSL

Domain: <https://attendance.yourcollege.edu>

---

## **3. Core Features**

### **3.1 Device Registration (REVISED - Explicit Consent)**

**Problem from v2.0:** Silent auto-registration causes device limit issues.

**FIXED Implementation:**

**Backend - Track Device Count:**

@Service

```
public class DeviceService {
```

```
    private static final int MAX_DEVICES_PER_USER = 5;
```

```
    public DeviceRegistrationResponse checkDeviceStatus(Long userId, String potentialToken) {
```

```
        // Check if token exists
```

```
        if (potentialToken != null) {
```

```
            Optional<Device> device = deviceRepository.findByUserIdAndToken(userId, potentialToken);
```

```
            if (device.isPresent() && device.get().isActive()) {
```

```
                return DeviceRegistrationResponse.existing(device.get());
```

```
            }
```

```
        }
```

```
        // Token missing or invalid - need to register
```

```
        long currentDevices = deviceRepository.countByUserIdAndIsActive(userId, true);
```

```
        if (currentDevices >= MAX_DEVICES_PER_USER) {
```

```
            return DeviceRegistrationResponse.limitReached(currentDevices);
```

```
        }
```

```

        return DeviceRegistrationResponse.needsRegistration(currentDevices);
    }

    public Device registerNewDevice(Long userId, DeviceInfo info) {
        long currentDevices = deviceRepository.countByUserIdAndIsActive(userId, true);

        if (currentDevices >= MAX_DEVICES_PER_USER) {
            throw new DeviceLimitException(
                "Maximum " + MAX_DEVICES_PER_USER + " devices allowed. " +
                "Please remove an old device from Settings."
            );
        }

        String token = UUID.randomUUID().toString();

        Device device = new Device();
        device.setUserId(userId);
        device.setToken(token);
        device.setDeviceName(info.getName());
        device.setUserAgent(info.getUserAgent());
        device.setRegisteredAt(LocalDate.now());
        device.setIsActive(true);

        return deviceRepository.save(device);
    }
}

```

### Frontend - Explicit Registration Modal:

```

function DeviceRegistrationModal({ open, onClose, deviceCount, maxDevices }) {
    const [deviceName, setDeviceName] = useState("");
    const [loading, setLoading] = useState(false);

```

```
const detectDeviceName = () => {  
  const ua = navigator.userAgent;  
  if (/iPhone/.test(ua)) return "My iPhone";  
  if (/Android/.test(ua)) return "My Android Phone";  
  if (/iPad/.test(ua)) return "My iPad";  
  if (/Mac/.test(ua)) return "My MacBook";  
  if (/Windows/.test(ua)) return "My Windows PC";  
  return "My Device";  
};
```

```
useEffect(() => {  
  setDeviceName(detectDeviceName());  
}, []);
```

```
const handleRegister = async () => {  
  setLoading(true);  
  try {  
    const response = await api.post('/auth/register-device', {  
      name: deviceName,  
      userAgent: navigator.userAgent  
    });  
  
    // Save token in multiple places  
    const token = response.data.token;  
    localStorage.setItem('device_token', token);  
    sessionStorage.setItem('device_token', token);  
    document.cookie = `device_token=${token}; max-age=31536000; path=/; SameSite=Strict`;  
  
    toast.success('Device registered successfully!');  
    onClose(true); // true = success
```

```

} catch (error) {
  if (error.response?.status === 400) {
    toast.error('Device limit reached. Remove old devices from Settings.');
```

}

```

  } else {
    toast.error('Failed to register device');
```

}

```

  } finally {
    setLoading(false);
  }
};

return (
  <Dialog open={open} onClose={() => {}} disableEscapeKeyDown>
    <DialogTitle>
      New Device Detected
    </DialogTitle>
    <DialogContent>
      <Alert severity="info" sx={{ mb: 2 }}>
        This device is not registered. You need to register it to mark attendance.
      </Alert>

      <Typography variant="body2" gutterBottom>
        Device Slots Used: <strong>{deviceCount} / {maxDevices}</strong>
      </Typography>

      <TextField
        fullWidth
        label="Device Name"
        value={deviceName}
        onChange={(e) => setDeviceName(e.target.value)}

```

```

        margin="normal"

        helperText="Give this device a recognizable name"
    />

    <Alert severity="warning" sx={{ mt: 2 }}>
        You can register up to {maxDevices} devices. Manage devices in Settings.
    </Alert>
</DialogContent>
<DialogActions>
    <Button onClick={() => onClose(false)} disabled={loading}>
        Cancel
    </Button>
    <Button
        onClick={handleRegister}
        variant="contained"
        disabled={loading || !deviceName.trim()}
    >
        {loading ? <CircularProgress size={24} /> : 'Register Device'}
    </Button>
</DialogActions>
</Dialog>
);
}

```

#### **App.js - Check Device on Login:**

```

function App() {
    const [showDeviceModal, setShowDeviceModal] = useState(false);
    const [deviceInfo, setDeviceInfo] = useState(null);

    useEffect(() => {
        checkDeviceStatus();
    }, []);

```

```

const checkDeviceStatus = async () => {

  const token = getToken(); // Check localStorage, sessionStorage, cookie

  try {

    const response = await api.post('/auth/check-device', { token });

    if (response.data.status === 'NEEDS_REGISTRATION') {

      setDeviceInfo({

        currentCount: response.data.deviceCount,

        maxDevices: response.data.maxDevices

      });

      setShowDeviceModal(true);

    } else if (response.data.status === 'LIMIT_REACHED') {

      toast.error('Device limit reached. Remove old devices from Settings.');

```

```

onClose={() => {
  setShowDeviceModal(false);
  if (success) {
    // Reload or continue to app
    window.location.reload();
  }
}}
deviceCount={deviceInfo?.currentCount || 0}
maxDevices={deviceInfo?.maxDevices || 5}
/>
</>
);
}

```

#### Benefits:

- ☒ User is aware of device registration
- ☒ Sees slot usage (3/5 devices)
- ☒ Can manage devices in Settings
- ☒ Won't accidentally hit limit

### 3.2 IP Validation with Mobile Data Warning

**Problem from v2.0:** Students on 4G/5G bypass college network.

#### FIXED Implementation:

##### Pre-Scan Connectivity Check:

```

function QRScanner() {
  const [networkCheck, setNetworkCheck] = useState(null);
  const [canScan, setCanScan] = useState(false);

  useEffect(() => {
    checkNetworkConnectivity();
  }, []);
}

```

```

const checkNetworkConnectivity = async () => {
  try {
    const response = await api.get('/attendance/check-network');

    if (response.data.onCampusNetwork) {
      setNetworkCheck({ status: 'success', message: 'Connected to campus network ✓' });
      setCanScan(true);
    } else {
      setNetworkCheck({
        status: 'error',
        message: 'Not on campus network. Please connect to College Wi-Fi and disable Mobile Data.'
      });
      setCanScan(false);
    }
  } catch (error) {
    setNetworkCheck({
      status: 'warning',
      message: 'Unable to verify network. Make sure you are connected to College Wi-Fi.'
    });
    setCanScan(true); // Allow attempt, but warn
  }
};

return (
  <Card>
    <CardContent>
      {/* Network Status Banner */}
      {networkCheck && (
        <Alert
          severity={networkCheck.status}

```

```

sx={{ mb: 2 }}
action={
  <Button size="small" onClick={checkNetworkConnectivity}>
    Retry
  </Button>
}
>
{networkCheck.message}
</Alert>
)}

```

```

{/* Instructions */}
<Alert severity="info" sx={{ mb: 2 }}>
  <Typography variant="subtitle2" gutterBottom>
    Before scanning:
  </Typography>
  <ul style={{ margin: 0, paddingLeft: 20 }}>
    <li>Connect to <strong>College Wi-Fi</strong></li>
    <li>Turn OFF <strong>Mobile Data (4G/5G)</strong></li>
    <li>Allow <strong>Location Access</strong></li>
  </ul>
</Alert>

```

```

{/* Scan Button */}
<Button
  variant="contained"
  size="large"
  fullWidth
  disabled={!canScan}
  onClick={() => setScanning(true)}
>

```

Scan QR Code

</Button>

```
{/* Troubleshooting */}
```

```
{!canScan && (
```

```
<Box mt={2}>
```

```
<Typography variant="caption" color="text.secondary">
```

```
<strong>Troubleshooting:</strong><br/>
```

- Check if connected to college Wi-Fi<br/>
- Disable mobile data in phone settings<br/>
- Refresh this page after connecting

```
</Typography>
```

```
</Box>
```

```
}}
```

```
</CardContent>
```

```
</Card>
```

```
);
```

```
}
```

### Backend - Helpful Error Messages:

```
@PostMapping("/attendance/mark")
```

```
public AttendanceResponse markAttendance(
```

```
@RequestBody AttendanceRequest request,
```

```
HttpServletRequest httpRequest
```

```
) {
```

```
String clientIP = getClientIP(httpRequest);
```

```
if (!ipValidationService.isFromCampusNetwork(clientIP)) {
```

```
// Helpful error message
```

```
throw new ValidationException(
```

```
"Not on campus network.
```

```

" +
    "Your IP: " + clientIP + "
" +
    "Required: College Wi-Fi

" +
    "Steps to fix:
" +
    "1. Connect to College Wi-Fi
" +
    "2. Turn OFF Mobile Data (4G/5G)
" +
    "3. Refresh the page and try again"
);
}

```

```

// ... continue validation
}

```

#### **Backend - Network Check Endpoint:**

```

@GetMapping("/attendance/check-network")
public NetworkCheckResponse checkNetwork(HttpServletRequest request) {
    String clientIP = getClientIP(request);
    boolean onCampus = ipValidationService.isFromCampusNetwork(clientIP);

    return NetworkCheckResponse.builder()
        .onCampusNetwork(onCampus)
        .clientIP(clientIP)
        .message(onCampus ?
            "Connected to campus network" :
            "Not on campus network - connect to College Wi-Fi")
        .build();
}

```

```
}
```

---

### 3.3 GPS Validation with Passive Review (No Spam)

**Problem from v2.0:** 15 students with low confidence → 15 alerts → teacher annoyed.

**FIXED Implementation:**

**No Active Notifications - Just Color Coding:**

```
function SessionReport({ sessionId }) {  
  const [attendance, setAttendance] = useState([]);  
  
  const getConfidenceColor = (confidence, distance) => {  
    if (confidence === 'HIGH') return 'success.light';  
    if (confidence === 'LOW') return 'warning.light';  
    return 'default';  
  };  
  
  return (  
    <TableContainer>  
      <Table>  
        <TableHead>  
          <TableRow>  
            <TableCell>Student Name</TableCell>  
            <TableCell>Time</TableCell>  
            <TableCell>GPS Distance</TableCell>  
            <TableCell>Status</TableCell>  
          </TableRow>  
        </TableHead>  
        <TableBody>  
          {attendance.map(record => (  
            <TableRow  
              key={record.id}  
              sx={{
```

```

        backgroundColor: getConfidenceColor(record.confidence, record.distance)
    }}
>
<TableCell>{record.studentName}</TableCell>
<TableCell>{formatTime(record.markedAt)}</TableCell>
<TableCell>
    {record.distance.toFixed(0)}m
    {record.confidence === 'LOW' && (
        <Chip
            label="Review"
            size="small"
            color="warning"
            sx={{ ml: 1 }}
        />
    )}
</TableCell>
<TableCell>
    <Chip
        label="Present"
        color={record.confidence === 'HIGH' ? 'success' : 'warning'}
        size="small"
    />
</TableCell>
</TableRow>
    )))
</TableBody>
</Table>

{/* Summary at bottom */}
<Box mt={2} p={2} bgcolor="grey.100">
    <Typography variant="body2">

```

<strong>Summary:</strong><br/>

High Confidence: {attendance.filter(a => a.confidence === 'HIGH').length}<br/>

Low Confidence (Review): {attendance.filter(a => a.confidence === 'LOW').length}

</Typography>

</Box>

</TableContainer>

);

}

#### **Backend - Remove Notification Trigger:**

// BEFORE (v2.0 - causes spam):

```
if (locationResult.getConfidence().equals("LOW")) {  
    notificationService.alertTeacher(...); // ✗ Sends 15 alerts  
}
```

// AFTER (v2.1 - passive):

// Just save the confidence level, no alerts

```
attendance.setConfidence(locationResult.getConfidence());  
attendance.setDistanceMeters(locationResult.getDistance());  
attendanceRepository.save(attendance);
```

// Teacher sees yellow highlighting in report ✓

#### **Summary Stats (End of Class):**

```
function AttendanceSummary({ sessionId }) {
```

```
    const [stats, setStats] = useState(null);
```

```
    return (
```

```
        <Card>
```

```
            <CardContent>
```

```
                <Typography variant="h6" gutterBottom>
```

```
                    Session Summary
```

```
                </Typography>
```

```

<Grid container spacing={2}>
  <Grid item xs={4}>
    <Paper sx={{ p: 2, bgcolor: 'success.light' }}>
      <Typography variant="h4">{stats?.highConfidence}</Typography>
      <Typography variant="caption">High Confidence</Typography>
    </Paper>
  </Grid>

```

```

<Grid item xs={4}>
  <Paper sx={{ p: 2, bgcolor: 'warning.light' }}>
    <Typography variant="h4">{stats?.lowConfidence}</Typography>
    <Typography variant="caption">Low Confidence</Typography>
  </Paper>
</Grid>

```

```

<Grid item xs={4}>
  <Paper sx={{ p: 2, bgcolor: 'error.light' }}>
    <Typography variant="h4">{stats?.absent}</Typography>
    <Typography variant="caption">Absent</Typography>
  </Paper>
</Grid>
</Grid>

```

```

{stats?.lowConfidence > 0 && (
  <Alert severity="info" sx={{ mt: 2 }}>
    {stats.lowConfidence} students marked attendance from 30-100m distance
    (possibly GPS drift). Review yellow-highlighted rows above.
  </Alert>
)}
</CardContent>
</Card>

```

```
);  
}
```

#### Benefits:

- ☒ No notification spam
  - ☒ Teacher reviews passively after class
  - ☒ Visual (color-coded) rather than intrusive
  - ☒ Still highlights potential issues
- 

## 4. Validation Pipeline (Final)

### 4.1 Complete Implementation

```
@Service
```

```
@Transactional
```

```
public class AttendanceValidationService {
```

```
    @Autowired
```

```
    private DeviceService deviceService;
```

```
    @Autowired
```

```
    private QRTokenService qrTokenService;
```

```
    @Autowired
```

```
    private EnrollmentRepository enrollmentRepository;
```

```
    @Autowired
```

```
    private IPValidationService ipValidationService;
```

```
    @Autowired
```

```
    private LocationValidationService locationService;
```

```
    @Autowired
```

```
    private AttendanceRepository attendanceRepository;
```

```

public AttendanceResponse markAttendance(
    Long studentId,
    String deviceToken,
    AttendanceRequest request,
    HttpServletRequest httpRequest
){
    try {
        String clientIP = getClientIP(httpRequest);

        // Layer 1: Device Token
        Device device = deviceService.validateDeviceToken(studentId, deviceToken);

        // Layer 2: QR Token (20 second expiry)
        AttendanceSession session = qrTokenService.validateAndGetSession(request.getQrToken());

        // Layer 3: Enrollment Check
        if (!enrollmentRepository.existsByStudentIdAndSectionId(studentId, session.getSectionId())) {
            throw new ValidationException(
                "You are not enrolled in " + session.getSection().getCourse().getName()
            );
        }

        // Layer 4: IP Validation
        if (!ipValidationService.isFromCampusNetwork(clientIP)) {
            throw new ValidationException(
                "Not on campus network.
" +
                "Please connect to College Wi-Fi and disable Mobile Data."
            );
        }
    }
}

```

```
// Layer 5: GPS Validation (Soft Fail)

LocationValidationResult location = locationService.validateLocation(
    request.getLatitude(),
    request.getLongitude(),
    session.getClassroom()
);

if (!location.isValid()) {
    throw new ValidationException(
        String.format("You are %.0fm away from campus. Maximum: 100m",
            location.getDistance())
    );
}

// Layer 6: Duplicate Check

if (attendanceRepository.existsByStudentIdAndSessionId(studentId, session.getId())) {
    throw new ValidationException("Already marked attendance for this session");
}

// Layer 7: Device Reuse Check

if (attendanceRepository.existsBySessionIdAndDeviceToken(session.getId(), deviceToken)) {
    throw new ValidationException("Device already used for this session");
}

// ALL VALIDATIONS PASSED - Save

Attendance attendance = new Attendance();
attendance.setStudentId(studentId);
attendance.setSessionId(session.getId());
attendance.setDeviceToken(deviceToken);
attendance.setLatitude(request.getLatitude());
```

```

attendance.setLongitude(request.getLongitude());
attendance.setClientIP(clientIP);
attendance.setConfidence(location.getConfidence()); // HIGH or LOW
attendance.setDistanceMeters(location.getDistance());
attendance.setMarkedAt(LocalDateTime.now());

attendanceRepository.save(attendance);

// Update device last used
device.setLastUsed(LocalDateTime.now());
deviceService.save(device);

return AttendanceResponse.success(
    "Attendance marked successfully",
    location.getConfidence()
);

} catch (ValidationException e) {
    log.warn("Validation failed for student {}: {}", studentId, e.getMessage());
    return AttendanceResponse.error(e.getMessage());
} catch (Exception e) {
    log.error("Error marking attendance", e);
    return AttendanceResponse.error("System error. Please try again.");
}
}

private String getClientIP(HttpServletRequest request) {
    String ip = request.getHeader("X-Forwarded-For");
    if (ip == null || ip.isEmpty()) {
        ip = request.getHeader("X-Real-IP");
    }
}

```

```
    if (ip == null || ip.isEmpty()) {  
        ip = request.getRemoteAddr();  
    }  
    return ip.split(",")[0].trim();  
}  
}
```

---

## 5. Database Design (Final)

-- Updated attendance table with new fields

```
CREATE TABLE attendance (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    student_id BIGINT NOT NULL,  
    session_id BIGINT NOT NULL,  
    device_token VARCHAR(255) NOT NULL,  
    latitude DECIMAL(10, 8),  
    longitude DECIMAL(11, 8),  
    client_ip VARCHAR(45), -- Track IP address  
    confidence ENUM('HIGH', 'LOW') DEFAULT 'HIGH', -- GPS confidence  
    distance_meters DECIMAL(8, 2), -- Distance from classroom  
    marked_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (student_id) REFERENCES user(id),  
    FOREIGN KEY (session_id) REFERENCES attendance_session(id),  
    UNIQUE KEY unique_attendance (student_id, session_id),  
    INDEX idx_confidence (confidence), -- For filtering low confidence  
    INDEX idx_session_device (session_id, device_token)  
) ENGINE=InnoDB;
```

---

## 6. Implementation Guide

### 6.1 Week 1: Setup + HTTPS

#### Day 1-2: Project Setup

# Backend

```
spring init --dependencies=web,security,data-jpa,mysql attendance-backend
```

```
cd attendance-backend
```

```
# Frontend
```

```
npx create-react-app attendance-frontend
```

```
cd attendance-frontend
```

```
# Install Ngrok
```

```
npm install -g ngrok
```

### **Day 3-4: HTTPS Setup**

```
# Terminal 1: Start React
```

```
cd attendance-frontend
```

```
npm start # http://localhost:3000
```

```
# Terminal 2: Create HTTPS tunnel
```

```
ngrok http 3000
```

```
# Copy HTTPS URL: https://xyz.ngrok.io
```

### **Day 5-7: Basic Authentication**

- JWT setup
- Login page
- Test on phone via Ngrok URL

### **6.2 Week 2: Device Registration**

- Implement explicit device registration modal
- Add device management page
- Test clearing cache/cookies (should show modal)

### **6.3 Week 3: QR System**

- QR token generation
- Time-rotation mechanism
- QR scanner with geolocation

### **6.4 Week 4: IP Validation**

**Get College IP Ranges:**

# Talk to college IT department

# Ask for: "What are our public IP ranges?"

# Example response: "49.205.xxx.0/24, 10.0.0.0/8"

**Configure in application.properties:**

app.allowed-ip-ranges=49.205.xxx.0/24,10.0.0.0/8,172.16.0.0/12

## **6.5 Week 5-6: Complete Features**

- GPS validation with soft-fail
- Reports with color coding
- Export to CSV

## **6.6 Week 7-8: Testing + Deployment**

- Test with classmates
- Fix bugs
- Deploy to college server

---

## **7. Testing Strategy**

### **7.1 Testing Checklist**

#### **HTTPS Geolocation:**

- ☐ Works on localhost
- ☐ Works via Ngrok HTTPS URL
- ☐ Works on production HTTPS domain
- ☐ Error message if HTTP (non-localhost)

#### **Mobile Data:**

- ☐ Detects when on 4G/5G
- ☐ Shows helpful error message
- ☐ Pre-scan network check works

#### **Device Registration:**

- ☐ Modal shows on first use
- ☐ Shows device count (X/5)
- ☐ Can't silently hit limit
- ☐ Device management page works

#### **GPS Confidence:**

- [ ] High confidence: within 30m
  - [ ] Low confidence: 30-100m (yellow highlight)
  - [ ] Rejected: beyond 100m
  - [ ] No notification spam
- 

## 8. Deployment

### 8.1 Production Deployment (College Server)

#### Requirements from IT Department:

1. Server with Ubuntu/CentOS
2. Domain name (e.g., attendance.yourcollege.edu)
3. IP ranges for validation
4. MySQL database

#### Setup:

# Install Docker

```
sudo apt update
```

```
sudo apt install docker.io docker-compose
```

# Clone repository

```
git clone https://github.com/yourname/attendance-system
```

```
cd attendance-system
```

# Configure environment

```
cp .env.example .env
```

# Edit .env with college IP ranges, DB password, etc.

# Start services

```
docker-compose up -d
```

# Get SSL certificate

```
sudo certbot --nginx -d attendance.yourcollege.edu
```

---

## 9. Timeline (8 Weeks)

**Week 1:** Setup + HTTPS (Ngrok)

**Week 2:** Authentication + Device Registration

**Week 3:** QR System

**Week 4:** IP Validation

**Week 5:** GPS + Reports

**Week 6:** UI Polish

**Week 7:** Testing

**Week 8:** Deployment

---

## 10. Troubleshooting Guide

### 10.1 Common Issues

#### "Geolocation not working"

- Check if HTTPS (not HTTP)
- Use Ngrok during development
- Check browser console for errors

#### "Not on campus network" error

- Verify student on college Wi-Fi
- Check if mobile data is OFF
- Verify IP ranges with IT department

#### "Device not registered" after cache clear

- Expected behavior
- Modal should appear
- Student re-registers explicitly

#### GPS showing 50m+ indoors

- Normal GPS drift in buildings
  - System allows up to 100m
  - Marks as "low confidence" (yellow)
- 

## Appendix A: Professor Feedback Changelog

### Version 2.0 → 2.1 Changes

#### Issue 1: HTTPS Geolocation

- Added Ngrok setup instructions

- Self-signed certificate guide
- Production SSL setup

#### **Issue 2: Mobile Data IP**

- Pre-scan network check
- UI warnings to disable mobile data
- Helpful error messages

#### **Issue 3: Silent Registration**

- Removed auto-registration
- Explicit modal with device count
- Device management UI

#### **Issue 4: Notification Spam**

- Removed active alerts
- Color-coded reports (yellow = review)
- Passive teacher review

---

#### **DOCUMENT STATUS:**

-  Professor Approved
-  All Technical Issues Addressed
-  Ready for Development

**Good luck with your implementation!** 